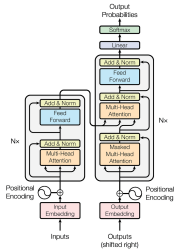
Advanced Artificial Intelligence and Machine Learning

Attention & Transformers



Learning goals

- Attention's two historical roots: Neural Turing Machines and the encoder-decoder bottleneck
- Tokenization: word, subword, and character level
- Self-attention, multi-head attention, and positional encoding
- The Transformer block and decoder: residuals, layer norm, masking, cross-attention
- Parameter count, efficient attention, and Transformer-XL
- Encoder-only, decoder-only, and encoder-decoder families

Why Attention? Two Historical Roots

WHY ATTENTION?

- In every operation we have seen so far — fully-connected layers, convolutions, RNN recurrence — how much a value in the input contributes to a value in the output is fixed by their positions, not by what they actually contain.
- Some tasks need more than that: e.g. in translation, a pronoun late in a sentence may need to agree in gender with a noun introduced far earlier.
- This motivated **attention**: let the network dynamically decide, based on content, which parts of a signal are relevant to which other parts — and route information between them directly, regardless of distance.
- Attention has two largely independent historical roots that later converged into the same mechanism: an external **memory** for neural networks, and a **bottleneck** fix for encoder-decoder RNNs.

POSITION-FIXED VS. CONTENT-BASED WEIGHTING

- **Fully-connected layer:** $y_i = \sum_j W_{ij} x_j$. The weight W_{ij} connecting input slot j to output slot i is a fixed number, learned once during training. Feed in a completely different input and the *same* W_{ij} still combines position j into position i — the weighting pattern is indexed by position, not content.
- **Convolution:** $y_i = \sum_k W_k \, x_{i+k}$. The kernel weight W_k depends only on the *relative offset* k between input and output position. A pixel 3 positions away always gets weighted by the same W_{-3}, whether it happens to be sky, an edge, or a face.

POSITION-FIXED VS. CONTENT-BASED WEIGHTING / 2

- **RNN recurrence:** $h_t = f(\mathrm{W}h_{t-1} + \mathrm{U}x_t)$. Information from k steps ago only reaches the current step by passing through the same matrix W, k times in a row — the path between two positions is fixed by their distance, never by whether their content makes them relevant to each other.
- In all three cases: swap the input values and the weighting pattern between positions stays identical. Only the architecture (which index connects to which) decides how much influence flows where.

ROOT 1: MEMORY — NEURAL TURING MACHINES

- **Graves et al. (2014)** proposed equipping a neural network with an explicit, differentiable external memory $\mathrm{M}_t \in \mathbb{R}^{N \times M}$ (N memory slots of dimension M), for long-term storage and retrieval beyond what a recurrent hidden state can hold.
- A "controller" (a feedforward or recurrent network) does not access memory at a fixed address; instead, at every step it computes a soft attention distribution over *all* memory slots and reads/writes a weighted combination of them.

ROOT 1: READING AND WRITING

- **Reading**: given attention weights $\vec{w}_t \in \mathbb{R}_+^N$ with $\sum_n w_t(n) = 1$,

$$\vec{r}_t = \sum_{n=1}^{N} w_t(n)\, \mathrm{M}_t(n).$$

- **Writing**: given weights $\vec{w}_t$, an erase vector $\vec{e}_t \in [0,1]^M$ and an add vector $\vec{a}_t \in \mathbb{R}^M$,

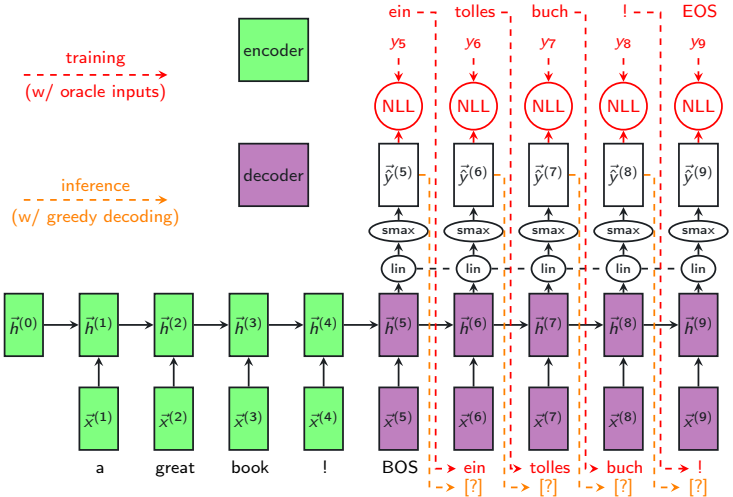
$$\forall n,\ \mathrm{M}_t(n) = \mathrm{M}_{t-1}(n)\,(1 - w_t(n)\vec{e}_t) + w_t(n)\vec{a}_t.$$

- The weights $\vec{w}_t$ are themselves computed by the controller from a **key vector** it emits, matched against the content of every memory slot — the same "query matched against keys" idea attention mechanisms still use today.

ROOT 1: WHY IT MATTERS HERE

- On synthetic tasks (copying a sequence, estimating N-gram statistics), a Neural Turing Machine solves problems a plain LSTM of the same size struggles with, because it can store and later retrieve specific past information instead of compressing everything into one hidden state.
- The mechanism that reappears throughout this session — a query matched against keys to retrieve a weighted combination of values — was already fully present here, applied to an external memory rather than to a sequence.
- The second root, covered next, arrives at the same idea from a completely different problem: how to translate a sentence without forcing all of it through one fixed-size vector.

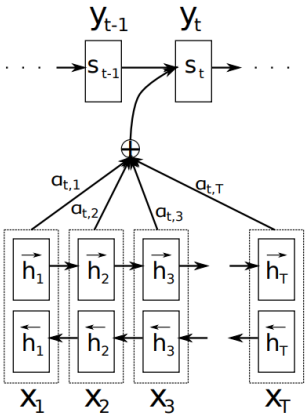
ROOT 2: ENCODER-DECODER RNNS



ROOT 2: THE FIXED-SIZE BOTTLENECK

- In an RNN, at a given point in time j, the information about all past inputs $x^{(1)} \dots x^{(j)}$ is "crammed" into the state vector $\vec{h}^{(j)}$ (and $\vec{c}^{(j)}$ for an LSTM)
- So for long sequences, the state becomes a **bottleneck**
- Especially problematic in encoder-decoder models (e.g., for Machine Translation)
- Solution: Attention ▸ Bahdanau et al., 2015 – an architectural modification of the RNN encoder-decoder that allows the model to "attend to" past encoder states

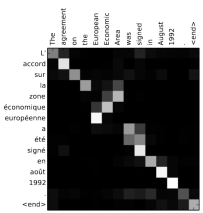
ROOT 2: BAHDANAU ATTENTION

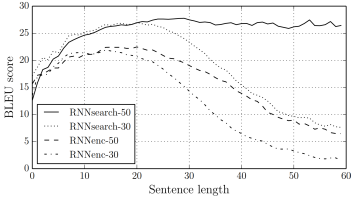


▸ Source: Bahdanau et al., 2015

- The encoder keeps *all* its states $\vec{h}_1 \ldots \vec{h}_T$ instead of only the last one.
- To produce output y_t, the decoder state s_t receives a weighted sum $\sum_j \alpha_{t,j} \vec{h}_j$ of all encoder states.
- The weights $\alpha_{t,j}$ are recomputed at every output step from how relevant $\vec{h}_j$ is to the current decoder state — the decoder chooses *where to look* at each step.

ROOT 2: ATTENTION IN PRACTICE





- **Left**: an attention (alignment) matrix for one translated sentence — brighter cells mean the decoder attended more strongly to that source word when generating a given target word.
- **Right**: BLEU score (translation quality; higher is better) vs. sentence length. `RNNenc` is the classical fixed-vector encoder-decoder; `RNNsearch` adds attention. The gap widens sharply on longer sentences — exactly where a single fixed-size context vector is the tightest bottleneck.

Source: Bahdanau et al. (2015).

The Transformer

MACHINE TRANSLATION

- Sequence-to-sequence task
- Already served as a motivation for introducing the "ordinary" Attention-mechanism ▸ Bahdanau et al., 2015
- Crucial, that the decoder has access to the whole input sequence
  $\rightarrow$ This is very well solved by cross-attention
- Good contextualization in the encoder improves translation quality
  - (Bidirectional) RNNs/LSTMs are only (concatenated) unidirectional architectures
  - Transformer-Encoder layers are bidirectional by construction
  - Stacking them on top of each other makes this bidirectional contextualization even "deeper"

WMT 2014 EN-TO-DE AND EN-TO-FR

Parallel training data:

Parallel data:

File	Size	CS-EN	DE-EN	HI-EN	FR-EN	RU-EN	Notes
Europarl v7	628MB	✓	✓		✓		same as previous year, corpus home page
Common Crawl corpus	876MB	✓	✓		✓	✓	same as previous year
UN corpus	2.3GB				✓		same as previous year, corpus home page
News Commentary	77MB	✓	✓		✓	✓	updated, data with document boundaries
10^9French-English corpus	2.3 GB				✓		same as previous year [md5|sha1]
CzEng 1.0	115MB	✓					same as previous year, corpus home page (avoid sections 98 and 99)
Yandex 1M corpus	121MB					✓	corpus home page; v1.3 now in original case
Wiki Headlines	7.8MB			✓		✓	Provided by CMU. The ru-en is unchanged from last year.
HindEnCorp	25MB			✓			Collected by Charles University
~~The JHU Corpus~~				~~✓~~			This is fully contained in HindEnCorp, so not made available here.

THE BLEU SCORE

- Based on n-gram overlap from candidate and reference sentence
- Precision (for each n-gram) calculated as follows:

$$P_n = \frac{\sum_{C \in \{ \text{ Candidates } \}} \sum_{n-gram \in C} \text{ Count }_{\text{clip}} (n- \text{ gram })}{\sum_{C' \in \{ \text{ Candidates } \}} \sum_{n-gram' \in C'} \text{ Count } (n- \text{ gram } ')}$$

- Finally, the BLEU score can be computed as

$$\texttt{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \cdot \log(P_n) \right),$$

- where BP is a "brevity penalty" to penalize short generations, N is the number of n-grams & w_n the weight for each P_n (usually $\frac{1}{N}$)

TRANSFORMER FOR MT

The Transformer ..

- .. outperforms the previous SOTA models
- .. at a lower number of required training FLOPs

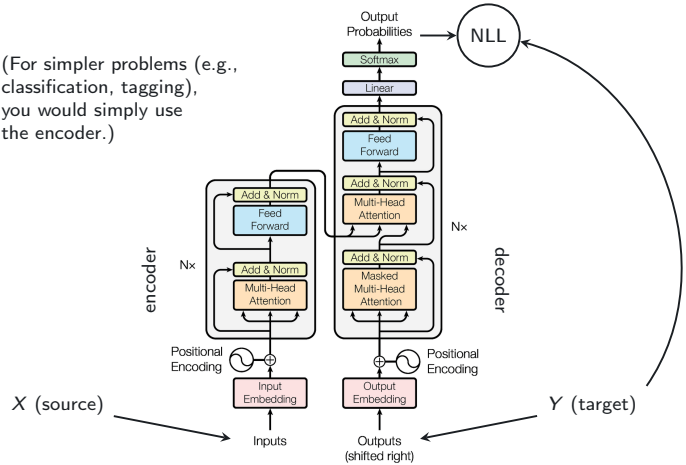
Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

▸ Source: Vaswani et al., 2017

THE TRANSFORMER ARCHITECTURE

(For simpler problems (e.g., classification, tagging), you would simply use the encoder.)



▸ Source: Vaswani et al., 2017

Tokenization

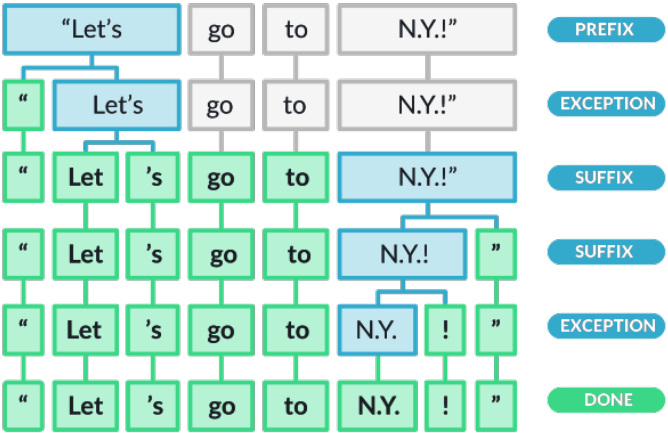
PROCESS OF TEXT TOKENIZATION

- The Transformer above consumes a sequence of "tokens". Tokenization is the preprocessing step that produces them: breaking raw text into smaller discrete units (letters, words, sub-words, ...) and encoding each unique unit with an integer ID.
- Performed once over an entire corpus, this yields a fixed **vocabulary** of unique tokens — the set of IDs the model's embedding lookup table must cover.
- A mandatory preprocessing step for essentially every NLP task, and the direct source of the token sequences fed into every Transformer frame in this session.

WHY TOKENIZE?

- Text must be turned into numbers before any neural network can process it; the question is only which granularity to encode.
- Whole documents are too large and unstructured to model directly — meaning is built bottom-up from smaller units.
- Tokenization also normalizes incidental formatting (extra whitespace, casing, punctuation spacing) that carries no meaning.

TOKENIZATION IN ACTION



Source: spaCy

TOKENIZATION TYPES

- **Word tokenization**: split on whitespace/punctuation — most common, but a fixed vocabulary can't cover every word that will ever appear.
- **Subword tokenization**: split words into smaller meaningful (or at least frequent) units — the standard choice for modern Transformers.
- **Character tokenization**: split into individual characters — tiny vocabulary, no out-of-vocabulary problem, but much longer sequences.
- (Paragraph, sentence, and whitespace tokenization also exist, as coarser or purely mechanical splits; rarely the final step for a Transformer's input.)

WORD, SUBWORD, AND CHARACTER TOKENIZATION

Word tokenization — whitespace and punctuation mark boundaries:

"What is the tallest building?" $\rightarrow$ *{What, is, the, tallest, building, ?}*

Simple and interpretable, but the vocabulary must contain every word form the model will ever see, or it falls back to an "unknown token" and loses information.

WORD, SUBWORD, AND CHARACTER TOKENIZATION / 2

Subword tokenization — split words into smaller, more frequent units (roots, prefixes, suffixes):

> *"What is the tallest building?" → {What, is, the, tall, est, build, ing, ?}*

Keeps the vocabulary small while still generalizing to unseen word forms — important for morphologically rich languages, and the choice essentially every modern Transformer makes.

WORD, SUBWORD, AND CHARACTER TOKENIZATION / 3

Character tokenization — split into individual characters:

$$\text{"What...?"} \rightarrow \{W, h, a, t, ..., ?\}$$

No out-of-vocabulary words are possible at all (any string is just a sequence of known characters), at the cost of much longer sequences — and, as we will see, self-attention's cost scales quadratically with sequence length.

Attention, Formalized

ATTENTION: THE BASIC RECIPE (1)

- **Ingredients:**

- One query vector: $\mathrm{q} \in \mathbb{R}^{d_q}$
- J key vectors: $\mathrm{K} \in \mathbb{R}^{J \times d_k}; (\vec{k}_1 \ldots \vec{k}_J)$
- J value vectors: $\mathrm{V} \in \mathbb{R}^{J \times d_v}; (\vec{v}_1 \ldots \vec{v}_J)$
- Scoring function $a : \mathbb{R}^{d_q} \times \mathbb{R}^{d_k} \to \mathbb{R}$
  - Maps a query-key pair to a scalar ("score")
  - a may be parametrized by parameters θ_a

ATTENTION: THE BASIC RECIPE (2)

- **Step 1**: Apply a to $\vec{q}$ and all keys $\vec{k}_j$ to get scores (one per key):

$$\vec{e} = \begin{bmatrix} e_1 \\ \vdots \\ e_J \end{bmatrix} = \begin{bmatrix} a(\vec{q}, \vec{k}_1; \theta_a) \\ \vdots \\ a(\vec{q}, \vec{k}_J; \theta_a) \end{bmatrix}$$

- **Step 2**: Turn e into a probability distribution with the softmax function

$$\alpha_j = \frac{\exp(e_j)}{\sum_{j'=1}^{J} \exp(e_{j'})}$$

  - Note that $\sum_j \alpha_j = 1$

ATTENTION: THE BASIC RECIPE (3)

- **Step 3**: α-weighted sum over $\vec{V}$ yields one d_v-dimensional output vector:

$$\vec{o} = \sum_{j=1}^{J} \alpha_j \vec{v}_j$$

  - Intuition: α_j is how much "attention" the model pays to $\vec{v}_j$ when computing $\vec{o}$.
- In a neural network, $\vec{q}$, $\vec{k}_j$ and $\vec{v}_j$ are functions of the input and of trainable parameters
- So the *model* learns which keys are relevant for which queries, based on the training data and loss function

ATTENTION IN THE TRANSFORMER

- We can use attention on many different "things", including:
  - The pixels of images
  - The nodes of knowledge graphs
  - The words of a vocabulary
- Here, we focus on scenarios where the query, key and value vectors represent tokens (e.g., words, characters, etc.) in sequences (e.g., sentences, paragraphs, etc.).

ATTENTION IN THE TRANSFORMER

Cross-Attention:

- Let $X = (x_1 \dots x_{J_x})$, $Y = (y_1 \dots y_{J_y})$ be two sequences (e.g., source and target in a sequence-to-sequence problem)
- The query vectors represent tokens in Y and the key/value vectors represent tokens in X ("Y attends to X")

Self-Attention:

- There is only one sequence $X = (x_1 \dots x_J)$
- The query, key and value vectors represent tokens in X ("X attends to itself")

CROSS-ATTENTION (1)

- We start by describing cross-attention, self-attention can easily be derived by assuming $\vec{X} = \vec{Y}$.
- Let $\vec{X} \in \mathbb{R}^{J_x \times d_x}$, $\vec{Y} \in \mathbb{R}^{J_y \times d_y}$ be representations of X, Y (e.g., stacked word embeddings, or the outputs of a previous layer)
- Let $\theta = \{\vec{W}^{(q)} \in \mathbb{R}^{d_y \times d_q}, \vec{W}^{(k)} \in \mathbb{R}^{d_x \times d_k}, \vec{W}^{(v)} \in \mathbb{R}^{d_x \times d_v}\}$ be trainable weight matrices
- We transform $\vec{Y}$ into a matrix of query vectors:

$$\vec{Q} = \vec{Y}\vec{W}^{(q)}$$

- We transform $\vec{X}$ into matrices of key and value vectors:

$$\vec{K} = \vec{X}\vec{W}^{(k)}; \qquad \vec{V} = \vec{X}\vec{W}^{(v)}$$

CROSS-ATTENTION (2)

- To calculate the e scores, Vaswani et al. use a parameter-less scaled dot product instead of Bahdanau's complicated FFN
- Step 1 of the basic recipe: *Scores*

$$e_{j,j'} = a(\vec{q}_j, \vec{k}_{j'}) = \frac{\vec{q}_j^T \vec{k}_{j'}}{\sqrt{d_k}}$$

- Note: This requires that $d_q = d_k$

CROSS-ATTENTION (3)

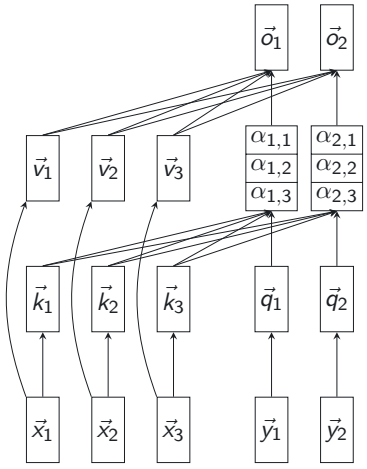
- Attention weights and outputs are defined like before
- Step 2 of the basic recipe: *Weights*

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J_x} \exp(e_{j,j''})}$$

- Step 3 of the basic recipe: *Outputs*

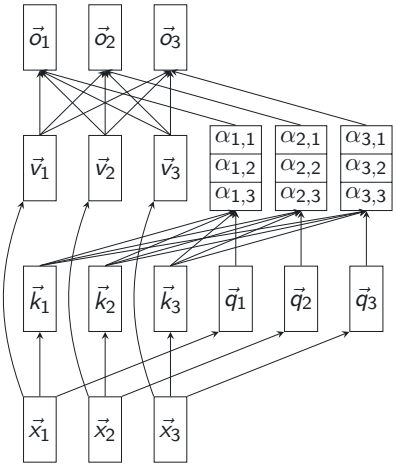
$$\vec{o}_j = \sum_{j'=1}^{J_x} \alpha_{j,j'} \vec{v}_{j'}$$

CROSS-ATTENTION (4)



Cross-attention

SELF-ATTENTION



Self-attention

SELF-ATTENTION FORMALIZED

- Let $\vec{X} \in \mathbb{R}^{J_x \times d_x}$ be a representation of X (e.g., stacked word embeddings, or the outputs of a previous layer)
- Let $\theta = \{\vec{W}^{(q)} \in \mathbb{R}^{d_x \times d_q}, \vec{W}^{(k)} \in \mathbb{R}^{d_x \times d_k}, \vec{W}^{(v)} \in \mathbb{R}^{d_x \times d_v}\}$ be trainable weight matrices
- We transform $\vec{X}$ into a matrix of query vectors:

$$\vec{Q} = \vec{X}\vec{W}^{(q)}$$

- And we transform $\vec{X}$ into matrices of key and value vectors:

$$\vec{K} = \vec{X}\vec{W}^{(k)}; \qquad \vec{V} = \vec{X}\vec{W}^{(v)}$$

- Execute Steps 1 – 3 of the basic recipe

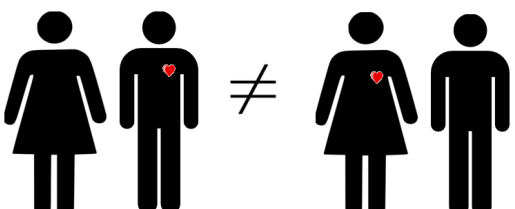
A GENERAL PROPERTY: PERMUTATION INVARIANCE/EQUIVARIANCE

- For a permutation σ of sequence positions, standard attention is:
  - **invariant** to a permutation of the keys and values: $Y(\vec{Q}, \sigma(\vec{K}), \sigma(\vec{V})) = Y(\vec{Q}, \vec{K}, \vec{V})$,
  - **equivariant** to a permutation of the queries: $Y(\sigma(\vec{Q}), \vec{K}, \vec{V}) = \sigma(Y(\vec{Q}, \vec{K}, \vec{V}))$.
- Consequently, self-attention (where $\vec{Q}, \vec{K}, \vec{V}$ all come from the same sequence) is equivariant to a permutation of its input: reorder the tokens and the outputs simply reorder the same way — **the values themselves don't change**.
- So self-attention alone cannot tell *where* in the sequence a token is, only *what* the other tokens are. The next frames show what goes wrong, and how position embeddings fix it.

Framing after Fleuret, *Deep Learning* (UNIGE) §13.2, CC BY-NC-SA 4.0.

CAN SELF-ATTENTION MODEL WORD ORDER?

- Hypothetical model:
  - simple word embedding lookup layer
  - self-attention layer
- Let $X^{(1)}$, $X^{(2)}$ be two sentences of the same length J, which contain the same words, but in a different order
- Example: "john loves mary" vs. "mary loves john"



CAN SELF-ATTENTION MODEL WORD ORDER?

- By the permutation equivariance above: reordering the input tokens only reorders the outputs, it never changes them.
- In other words: The representation of mary is identical to that of mary, and the representation of john is identical to that of john
- **Question:** Can the other layers in the Transformer architecture (feed-forward net, layer normalization) help with the problem?
  - No, because they apply the same function to all positions.
- **Question:** Would it help to apply more self-attention layers?
  - No. Since the representations of identical words are still identical in $\vec{O}$, the next self-attention layer will have the same problem.
- Luckily, we just need to ensure that input embeddings of identical words at different positions are not identical.

POSITION EMBEDDINGS

- Add to every input word embedding a position embedding $\vec{p} \in \mathbb{R}^d$:
- Input embedding of word "mary" in position j: $\vec{x}_j = \vec{w}_{\mathcal{I}(\text{mary})} + \text{p}_j$

$$\vec{w}_{\mathcal{I}(\text{mary})} + \vec{p}_j \neq \vec{w}_{\mathcal{I}(\text{mary})} + \vec{p}_{j'} \text{ if } j \neq j'$$

- Option 1 ▸ Source: Vaswani et al., 2017
  (Deterministic) Sinusoidal position embeddings:

$$p_{j,i} = \begin{cases} \sin\left(\frac{j}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{j}{10000^{\frac{i-1}{d}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

- Option 2 ▸ Source: Devlin et al., 2019
  Trainable position embeddings: $\vec{P} \in \mathbb{R}^{J^{\max} \times d}$
  (Disadvantage: Cannot deal with sentences longer than $J^{\max}$)

PARALLELIZED ATTENTION

- We want to apply our attention recipe to every query vector $\vec{q}_j$
- We could simply loop over all time steps $1 \leq j \leq J_x$ (or J_y) and calculate each $\vec{o}_j$ independently.
- Then stack all $\vec{o}_j$ into an output matrix $\vec{O} \in \mathbb{R}^{J_x \times d_v}$ (or $\mathbb{R}^{J_y \times d_v}$)
- But a loop does not use the GPU's capacity for parallelization, so it might be unnecessarily slow

PARALLELIZED SELF-ATTENTION

- Do some inputs (e.g., $\vec{q}_j$) depend on previous outputs (e.g., $\vec{o}_{j-1}$)? If not, we can parallelize the loop into a single function:

$$\vec{O} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{X}; \theta)$$

- Attention in Transformers is usually parallelizable, unless we are doing autoregressive inference (more on that later).

PARALLELIZED SELF-ATTENTION

- By the way: The Bahdanau model is not parallelizable in this way, because s_i (a.k.a. the query of the $i+1$'st step) depends on c_i (a.k.a. the attention output of the i'th step):

The hidden state s_i of the decoder given the annotations from the encoder is computed by

$$s_i = (1 - z_i) \circ s_{i-1} + z_i \circ \tilde{s}_i,$$

where

$$\begin{aligned}
\tilde{s}_i &= \tanh\left(W E y_{i-1} + U\left[r_i \circ s_{i-1}\right] + C c_i\right) \\
z_i &= \sigma\left(W_z E y_{i-1} + U_z s_{i-1} + C_z c_i\right) \\
r_i &= \sigma\left(W_r E y_{i-1} + U_r s_{i-1} + C_r c_i\right)
\end{aligned}$$

PARALLELIZED SELF-ATTENTION

- **Step 1**: The parallel application of the scaled dot product to all query-key pairs can be written as:

$$\vec{E} = \frac{\vec{Q}\vec{K}^T}{\sqrt{d_k}}; \quad \vec{E} \in \mathbb{R}^{J_x \times J_x}$$

$$\frac{1}{\sqrt{d_k}} \begin{bmatrix} - & \vec{q}_1 & - \\ & \vdots & \\ - & \vec{q}_{J_x} & - \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{k}_1 & \dots & \vec{k}_{J_x} \\ | & & | \end{bmatrix} = \begin{matrix} \\ \downarrow \\ \textit{queries} \\ \downarrow \end{matrix} \begin{matrix} \rightarrow \textit{keys} \rightarrow \\ \begin{bmatrix} e_{1,1} & \dots & e_{1,J_x} \\ \vdots & \ddots & \vdots \\ e_{J_x,1} & \dots & e_{J_x,J_x} \end{bmatrix} \end{matrix}$$

PARALLELIZED SCALED DOT PRODUCT SELF-ATTENTION

- **Step 2**: Softmax with normalization over the second axis (key axis):

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J_x} \exp(e_{j,j''})}$$

- Let's call this new normalized matrix $\vec{A} \in (0,1)^{J_x \times J_x}$
- The rows of $\vec{A}$, denoted $\vec{\alpha}_j$, are probability distributions (one $\vec{\alpha}_j$ per $\vec{q}_j$)

PARALLELIZED SCALED DOT PRODUCT SELF-ATTENTION

- **Step 3**: Weighted sum

$$\vec{O} = \vec{A}\vec{V}; \vec{O} \in \mathbb{R}^{J_x \times d_v}$$

$$\begin{bmatrix} - & \boldsymbol{\alpha}_1 & - \\ & \vdots & \\ - & \boldsymbol{\alpha}_{J_x} & - \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{v}_{:,1} & \dots & \vec{v}_{:,d_v} \\ | & & | \end{bmatrix} = \begin{matrix} \\ \downarrow \\ \textit{queries} \\ \downarrow \end{matrix} \begin{matrix} \overset{\rightarrow d_v (\text{value dims}) \rightarrow}{} \\ \begin{bmatrix} o_{1,1} & \dots & o_{1,d_v} \\ \vdots & \ddots & \vdots \\ o_{J_x,1} & \dots & o_{J_x,d_v} \end{bmatrix} \end{matrix}$$

... AS A ONE-LINER

$$\vec{O} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{X}; \theta) = \text{softmax}\Big(\frac{(\vec{X}\vec{W}^{(q)})(\vec{X}\vec{W}^{(k)})^T}{\sqrt{d_k}}\Big)(\vec{X}\vec{W}^{(v)})$$

- GPUs like matrix multiplications
  $\rightarrow$ usually a lot faster than RNN!
- But: The memory requirements of $\vec{E}$ and $\vec{A}$ are $\mathcal{O}(J_x^2)$
- Sequence lengths are an important issue.

ADD-ON: CHANGES FOR CROSS-ATTENTION

- **Step 1**: Scaled dot-product

$$\vec{E} = \frac{\vec{Q}\vec{K}^T}{\sqrt{d_k}}; \quad \vec{E} \in \mathbb{R}^{J_y \times J_x}$$

- **Step 2**: Softmax

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J_x} \exp(e_{j,j''})}$$

- **Step 3**: Output

$$\vec{O} = \vec{A}\vec{V}; \vec{O} \in \mathbb{R}^{J_y \times d_v}$$

- As one-liner:

$$\vec{O} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{Y}; \theta) = \text{softmax}\Big(\frac{(\vec{Y}\vec{W}^{(q)})(\vec{X}\vec{W}^{(k)})^T}{\sqrt{d_k}}\Big)(\vec{X}\vec{W}^{(v)})$$

MULTI-LAYER ATTENTION

- Sequential application of several attention layers, with separate parameters $\{\theta^{(1)} \dots \theta^{(N)}\}$
- In Transformer: sequential application of Transformer blocks
- There are some additional position-wise layers inside the Transformer block, i.e., $\vec{O}^{(n)}$ undergoes some additional transformations before becoming the input to the next Transformer block $n + 1$

MULTI-HEAD ATTENTION

- Application of several attention layers ("heads") in parallel
- M sets of parameters $\{\theta^{(1)}, \ldots, \theta^{(M)}\}$, with $\theta^{(m)} = \{\vec{W}^{(m,q)}, \vec{W}^{(m,k)}, \vec{W}^{(m,v)}\}$
- For every head, compute in parallel:

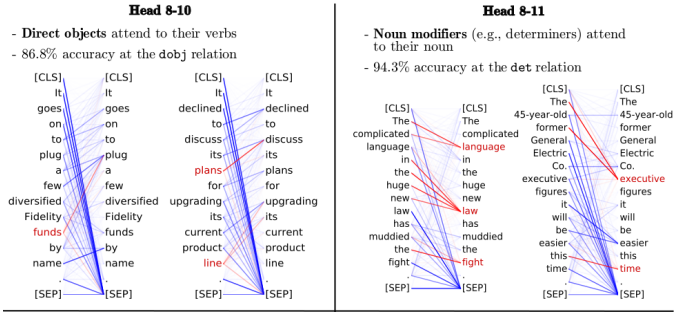
$$\vec{O}^{(m)} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{Y}; \theta^{(m)})$$

- Choice of dimensionality: $d_k = d_v = d_x/M$
- Concatenate all $\vec{O}^{(m)}$ along their last axis; then down-project the concatenation with an additional parameter matrix $\vec{W}^{(o)} \in \mathbb{R}^{Md_v \times d_x}$:

$$\vec{O} = [\vec{O}^{(1)}; \ldots; \vec{O}^{(M)}]\vec{W}^{(o)}$$

MULTI-HEAD ATTENTION

- Conceptually, multi-head attention is to single-head attention like a filter bank is to a single filter (CNNs)
- Division of labor: different heads model different kinds of inter-word relationships



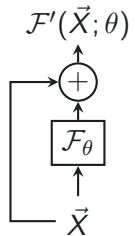
▸ Source: Clark et al., 2019

The Transformer Block

RESIDUAL CONNECTIONS

- Let $\mathcal{F}$ be a function with parameters θ
- $\mathcal{F}$ with a residual connection:

$$\mathcal{F}'(\vec{X};\theta) = \mathcal{F}(\vec{X};\theta) + \vec{X}$$



- Benefits: Information retention (we add to $\vec{X}$ but don't replace it)

LAYER NORMALIZATION

- Let $\theta = \{\vec{\gamma} \in \mathbb{R}^d, \vec{\beta} \in \mathbb{R}^d\}$ be trainable parameters
- Let $\vec{h} \in \mathbb{R}^d$ be an output vector of some layer (e.g., an $\vec{o}_j$ vector from an attention layer)
- Then layer normalization calculates:

$$\vec{\gamma} \odot \frac{\vec{h} - \mu}{\sigma} + \vec{\beta}$$

- where μ, σ are mean and standard deviation over the dimensions of $\vec{h}$:

$$\mu = \frac{1}{d} \sum_{i=1}^{d} h_i; \qquad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^{d} (h_i - \mu)^2}$$

- Benefits: Allows us to normalize vectors after every layer; helps against exploding activations on the forward pass
- In the Transformer, layer normalization is applied position-wise, i.e., every $\vec{o}_j$ is normalized independently

PRE-NORM VS. POST-NORM

- The original Transformer (Vaswani et al., 2017) normalizes *after* each sublayer's residual sum — "post-norm": $\vec{x}_{l+1} = \mathrm{LN}(\vec{x}_l + \mathcal{F}(\vec{x}_l))$, exactly what the previous two frames describe.
- Later work (Wang et al., 2019) found that normalizing *before* each sublayer instead — "pre-norm": $\vec{x}_{l+1} = \vec{x}_l + \mathcal{F}(\mathrm{LN}(\vec{x}_l))$ — keeps the residual path itself unnormalized all the way through the network, which stabilizes gradients enough to train much deeper Transformers without the learning-rate warm-up the original architecture needed.

PRE-NORM VS. POST-NORM / 2

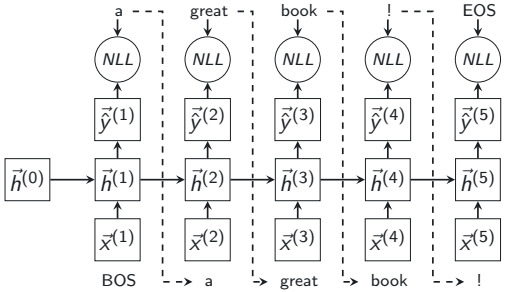
- Most modern implementations (e.g. GPT-style decoders) use pre-norm. We present post-norm here because it matches the original paper the rest of this section follows — the difference is a placement choice, not a change to the attention or feed-forward computations themselves.

Wang et al., *Learning Deep Transformer Models for Machine Translation* (2019); framing after Fleuret, *Deep Learning* (UNIGE) §13.3, CC BY-NC-SA 4.0.

The Decoder

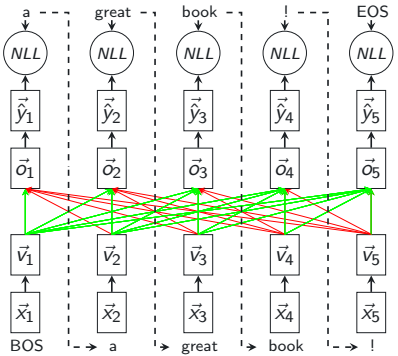
RNNS FOR AUTOREGRESSIVE LM & DECODING

- In autoregressive language modeling, or in the decoder of a sequence-to-sequence model, the task is to always predict the next word
- In an RNN, a given state $\vec{h}^{(j)}$ depends on past inputs $x^{(1)} \dots x^{(j)}$
- Thus, the RNN is unable to "cheat":



SELF-ATTENTION FOR AR LM & DECODING

- With attention, all $\vec{o}_j$ depend on all $\vec{v}_{j'}$ (and by extension, all $\vec{x}_{j'}$).
- This means that the model can easily cheat by looking at future words (red connections)



MASKED SELF-ATTENTION

- So when we use self-attention for language modeling or in a sequence-to-sequence decoder, we have to prevent $\vec{o}_j$ from attending to any $\vec{v}_{j'}$ where $j' > j$.
- **Question:** How can we do that?
- Remember:

$$\vec{o}_j = \sum_{j'=1}^{J} \alpha_{j,j'} \vec{v}_{j'}$$

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J} \exp(e_{j,j''})}$$

  - By hardcoding $e_{j,j'} = -\infty$ when $j' > j$ (in practice, "∞" is just a large constant)
  - That way, $\exp(e_{j,j'}) = \alpha_{j,j'} = 0$, so $\vec{v}_{j'}$ has no impact on $\vec{o}_j$

PARALLELIZED MASKED SELF-ATTENTION

- Step 1: Calculate $\vec{E}$ like we usually would
- Step 1B:

$$\vec{E}^{\text{masked}} = \vec{E} \odot \vec{M} + \infty\vec{M} - \infty; \qquad m_{j,j'} = \begin{cases} 1 & \text{if } j' \leq j \\ 0 & \text{otherwise} \end{cases}$$

- Example:

PARALLELIZED MASKED SELF-ATTENTION / 2

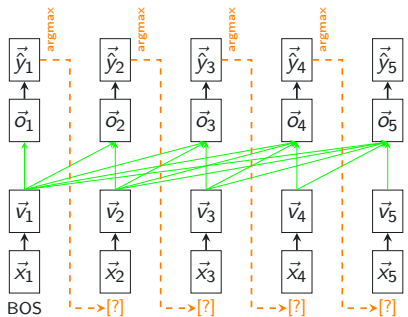
$$\vec{E} = \begin{bmatrix} e_{1,1} & e_{1,2} & e_{1,3} \\ e_{2,1} & e_{2,2} & e_{2,3} \\ e_{3,1} & e_{3,2} & e_{3,3} \end{bmatrix}; \qquad \vec{M} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\vec{E}^{\text{masked}} = \begin{bmatrix} e_{1,1} & -\infty & -\infty \\ e_{2,1} & e_{2,2} & -\infty \\ e_{3,1} & e_{3,2} & e_{3,3} \end{bmatrix}; \qquad \vec{A}^{\text{masked}} = \begin{bmatrix} 1 & 0 & 0 \\ \alpha_{2,1} & \alpha_{2,2} & 0 \\ \alpha_{3,1} & \alpha_{3,2} & \alpha_{3,3} \end{bmatrix}$$

$$\vec{o}_1 = \vec{v}_1; \qquad \vec{o}_2 = \alpha_{2,1}\vec{v}_1 + \alpha_{2,2}\vec{v}_2; \qquad \vec{o}_3 = \alpha_{3,1}\vec{v}_1 + \alpha_{3,2}\vec{v}_2 + \alpha_{3,3}\vec{v}_3$$

AR TRANSFORMER AT INFERENCE TIME

- During training (targets known): Use parallelized masked attention
- At inference time (targets unknown): Decode prediction in a loop
- Slower, but at least we don't have to worry about masking anymore



ENCODER-DECODER ATTENTION

Question: How do we connect encoder and decoder?

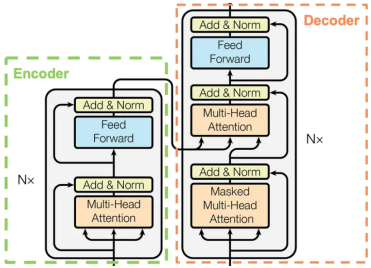
Construction of one decoder block:

1. Masked (Multi-Head) Attention layer (only target sequence)
2. "Ordinary" (Multi-Head) Attention layer
   - Queries from the previous decoder layer
   - Keys, Values from the encoder output
3. Feed-Forward layer (w/ residual connections & layer norm)

$\rightarrow$ Allows the decoder to attend to *all* tokens from the input sequences
(cf. ▸ Bahdanau et al., 2015 for RNNs)

Transformer Parameter Count

OVERVIEW OVER PARAMETER SOURCES

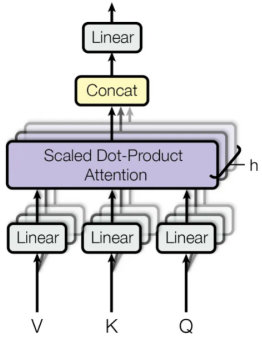


- **Encoder**
  - Multi-Head Attention
  - 2x Layernorm
  - Feed Forward Network
- **Decoder**
  - 2x Multi-Head Attention
  - 3x Layernorm
  - Feed Forward Network
- **Original sizes:**
  $d_{model} = 512$, $n_{head} = 8$, so
  $d_{qkv} = 512/8 = 64$

Derive the parameter count of each component, then sum them up!

▸ Vaswani et al., 2017

MULTI-HEAD ATTENTION

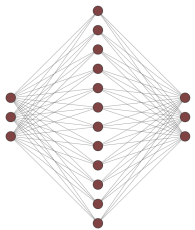


- Per head: $\boldsymbol{W_Q}, \boldsymbol{W_K}, \boldsymbol{W_V}$, each $d_{model} \times d_{qkv}$ (plus bias), with $d_{qkv} = d_{model}/n_{head}$
- Concatenating the n_{head} outputs gives d_{model} again; $\boldsymbol{W_O}$ is $d_{model} \times d_{model}$ (plus bias)

$$N_{attention} = \underbrace{(d_{model}^2 + d_{model})}_{W_O} + \underbrace{(d_{model} \cdot d_{qkv} + d_{qkv}) \cdot n_{head} \cdot 3}_{W_{Q,K,V} \text{ of all heads}}$$

$$= (d_{model}^2 + d_{model}) \cdot 4 \approx 4 \cdot d_{model}^2$$

FEED-FORWARD NETWORK AND LAYER NORM



- The FFN of the Transformer uses two fully connected layers, $d_{model} \to 4 \cdot d_{model} \to d_{model}$ (the factor 4 is a design decision by the authors of the paper)
- Weight matrices W_1: $d_{model} \times 4 \cdot d_{model}$ and W_2: $4 \cdot d_{model} \times d_{model}$, plus biases

$$\begin{aligned} N_{FFN} &= (4d_{model}^2 + 4d_{model}) + (4d_{model}^2 + d_{model}) \\ &= 8d_{model}^2 + 5d_{model} \approx 8d_{model}^2 \end{aligned}$$

Layer norm: one γ and one β per dimension (see the Layer Normalization frame), so $N_{Layernorm} = 2 \cdot d_{model}$.

TOTAL PARAMETER COUNT

$$N_{Encoder} = N_{attention} + N_{FFN} + 2 \cdot N_{Layernorm} = 12\, d_{model}^2 + 13\, d_{model}$$
$$N_{Decoder} = 2 \cdot N_{attention} + N_{FFN} + 3 \cdot N_{Layernorm} = 16\, d_{model}^2 + 19\, d_{model}$$

- Vanilla Transformer: $n_{Layers} = 6$ for both Encoder and Decoder, $d_{model} = 512$
- Exact:
  $6 \cdot (12 \cdot 512^2 + 13 \cdot 512) + 6 \cdot (16 \cdot 512^2 + 19 \cdot 512) = 44,138,496$
- Approximate: $6 \cdot (12 \cdot 512^2) + 6 \cdot (16 \cdot 512^2) = 44,040,192$

The count scales quadratically with d_{model}, so bias and layer norm terms are negligible — $28 \cdot d_{model}^2$ per encoder+decoder layer pair is a good approximation.

EMBEDDING PARAMETERS

We also have to consider the parameters of the embeddings!

- Let M be the maximum sequence length and V the size of the vocabulary
- We have $M \times d_{model}$ positional embedding parameters, which are fixed and not learned
- And $V \times d_{model}$ token embedding parameters, which are learned
- The original transformer has $V = 37000$ tokens:

$$N_{Embedding} = 37000 \cdot 512 = 18,944,000$$

- Adding those to the previous count yields:

$$N_{Final} = 18,944,000 + 44,040,192 = 62,984,192$$

Efficient Transformers

LIMITATION OF THE TRANSFORMER

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

not cool cool

Source: Vaswani et al. (2017)

Advantage:

- Every token can *directly* attend to each other token
- Cf. RNN: At worst n sequential operations (last to first token)

Severe Limitation:

- Every token attends to each other token (incl. itself)
  $\rightarrow$ We need to calculate n^2 attention weights
- Computational complexity of Transformer scales quadratically with the sequence length
  $\rightarrow$ Longer sequences are disproportionally expensive

EFFICIENT TRANSFORMERS

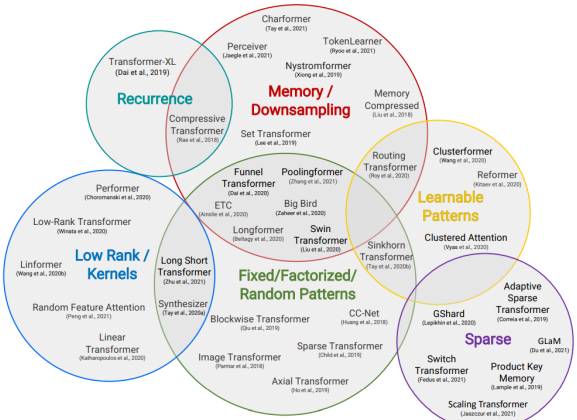
Broad overview on so-called "X-formers" ▸ Tay et al., 2020a

- Efficient & fast Transformer-based models
  $\rightarrow$ Reduce complexity from $\mathcal{O}(n^2)$ to (up to) $\mathcal{O}(n)$
- Claim on-par (or even) superior performance
- Different techniques used:
  - Fixed/Factorized/Random Patterns
  - Learnable Patterns (extension of the above)
  - Low-Rank approximations or Kernels
  - Recurrence (see e.g. ▸ Dai et al., 2019)
  - Memory modules

Side note:

- Most benchmark data sets not explicitly designed for evaluating long-range abilities of the models.
- Recently proposed: *Long Range Arena: A benchmark for efficient transformers* ▸ Tay et al., 2020b

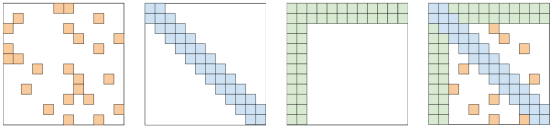
EFFICIENT TRANSFORMERS



▸ Tay et al., 2020a

SPARSE ATTENTION: FIXED PATTERNS

Idea: most query-key pairs contribute little, so compute only a fixed, sparse subset of the attention matrix (white cells = skipped). ▸ Source: Zaheer et al., 2020



(a) Random attention (b) Window attention (c) Global Attention (d) BIGBIRD

Figure 1: Building blocks of the attention mechanism used in BIGBIRD. White color indicates absence of attention. (a) random attention with $r = 2$, (b) sliding window attention with $w = 3$ (c) global attention with $g = 2$. (d) the combined BIGBIRD model.

- **(a) Random**: each query attends to a few randomly chosen keys
- **(b) Window (local)**: each query attends only to its neighbours, $|i - j| \leq w$ — the decoder's causal mask is the one-sided version of the same idea
- **(c) Global**: a few tokens attend to, and are attended by, every position — one cheap channel for long-range information
- **(d) BigBird** combines all three; Longformer (Beltagy et al., 2020) uses (b) + (c)

EFFICIENT TRANSFORMERS — TRADEOFF

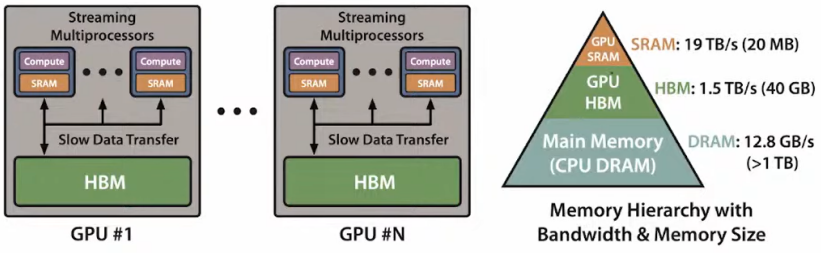
- *Quality:* How well can we approximate the results we would get when applying full attention?
- *Speed:* Can we reduce the computational complexity from quadratic to (nearly) linear?

A PRIMER ON FLASH ATTENTION

Compute-bound vs. memory-bound operations:

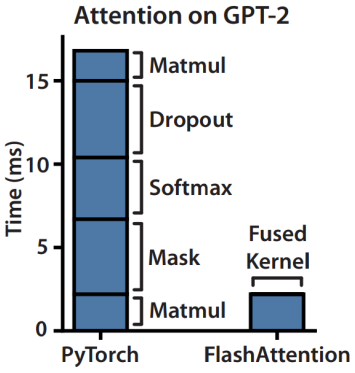
- Wall-clock time not (necessarily) a function of FLOPs
- Most of these "Efficient Transformers" focus on FLOP reduction
  $\rightarrow$ This ignores existing memory overheads
- *Compute-bound operations:* ($\approx$ FLOPs)
  - Matrix multiplication
- *Memory-bound operations:* ($\approx$ data transfer)
  - Attention

A PRIMER ON FLASH ATTENTION



▸ Source: Talk by Tri Dao, 2023

A PRIMER ON FLASH ATTENTION



▸ Source: Dao, 2022

- *Compute-bound operations:*
  - Matmul
- *Memory-bound operations:*
  - Mask
  - Dropout
  - Softmax

$\rightarrow$ Reducing memory-bound operations most effective for reducing wall-clock time

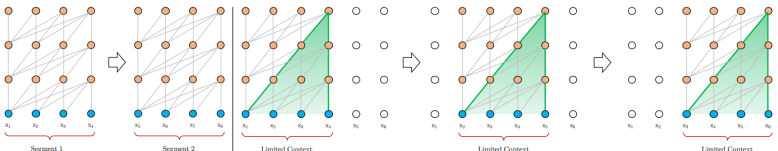
Long Sequences: Transformer-XL

TRANSFORMER-XL

Key facts:

- Objective: Autoregressive Language Modeling task
- Transformer decoder model
- Addresses long sequences
- Assumption: *No* infinite memory & compute; limited resources
- (Possible) Solution Vanilla Transformer:
  - Split corpus into shorter segments
  - Limited contextual information
- Solution Transformer-XL:
  - Segment-level recurrence mechanism
  - Able to model longer-term dependencies

TRANSFORMER-XL



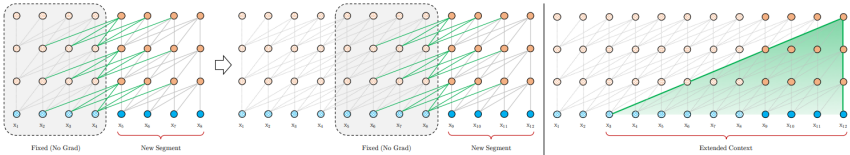
(a) Train phase. (b) Evaluation phase.

Figure 1: Illustration of the vanilla model with a segment length 4.

Source: Dai et al. (2019)

- Contextual information limited to segments
- Does not respect semantic or syntactic boundaries

TRANSFORMER-XL



(a) Training phase. (b) Evaluation phase.

Figure 2: Illustration of the Transformer-XL model with a segment length 4.

Source: Dai et al. (2019)

- Caches hidden states from the previous segment
- Contextual information flows across segments

SEGMENT RECURRENCE

- Let $s_\tau = [x_{\tau,1}, \ldots, x_{\tau,L}]$ and $s_{\tau+1} = [x_{\tau+1,1}, \ldots, x_{\tau+1,L}]$ be two consecutive segments of length L.
- Let $h_\tau^n \in \mathbb{R}^{L \times d}$ denote the n-th layer hidden states for s_τ.
- Using segment recurrence, the n-th layer hidden states for the following segment $s_{\tau+1}$ are computed as follows:

$$\tilde{h}_{\tau+1}^{n-1} = \mathit{Concat}[\mathit{SG}(h_\tau^{n-1}), h_{\tau+1}^{n-1}]$$

$$q_{\tau+1}^n = h_{\tau+1}^{n-1} W_q^\mathsf{T}; \quad k_{\tau+1}^n = \tilde{h}_{\tau+1}^{n-1} W_k^\mathsf{T}; \quad v_{\tau+1}^n = \tilde{h}_{\tau+1}^{n-1} W_v^\mathsf{T}$$

$$h_{\tau+1}^n = \mathit{Trafo}(q_{\tau+1}^n, k_{\tau+1}^n, v_{\tau+1}^n),$$

where $\mathit{SG}(\cdot)$ stands for "'stop-gradient"'.

RELATIVE POSITIONAL ENCODINGS

Problem:

- *Absolute* positional encodings (PE) would assign the same embedding to words in similar positions in both segments
- No positional difference between $x_{\tau,j}$ and $x_{\tau+1,j}$

Solution:

- Inject information about the relative distance between a query vector and the respective key vectors directly into the Attention mechanism
- *Comment:* Using relative PEs utterly necessary here, but also applicable independently of the segment recurrence

RELATIVE POSITIONAL ENCODINGS / 2

$$A_{i,j}^{\text{abs}} = \underbrace{E_{x_i}^\top W_q^\top W_k E_{x_j}}_{(a)} + \underbrace{E_{x_i}^\top W_q^\top W_k U_j}_{(b)} + \underbrace{U_i^\top W_q^\top W_k E_{x_j}}_{(c)} + \underbrace{U_i^\top W_q^\top W_k U_j}_{(d)}.$$

$$A_{i,j}^{\text{rel}} = \underbrace{E_{x_i}^\top W_q^\top W_{k,E} E_{x_j}}_{(a)} + \underbrace{E_{x_i}^\top W_q^\top W_{k,R} R_{i-j}}_{(b)} + \underbrace{u^\top W_{k,E} E_{x_j}}_{(c)} + \underbrace{v^\top W_{k,R} R_{i-j}}_{(d)}.$$

RELATIVE POSITIONAL ENCODINGS

Solution:

- Replace all absolute PEs with relative ones (fixed + sinusoidal)
  $\rightarrow R_{i-j}$ instead of U_j in (b) and (d)
- Replace all positional query vectors with single trainable embeddings
  $\rightarrow u$ and v instead of $U_i^\top W_q^\top$ in (c) and (d)
- Use separate weight matrices for linearly projecting E
  $\rightarrow W_{k,E}$ and $W_{k,R}$ instead of W_k

The Three Transformer Families

ENCODER-ONLY, DECODER-ONLY, ENCODER-DECODER

- You have now seen both halves of the original Transformer. In practice, most models use only one half, or both, depending on the task:

	Encoder-only	**Decoder-only**	**Encoder-decoder**
Attention	bidirectional	causal (masked)	both (+ cross-attn.)
Pretraining obj.	masked LM	next-token	varies (e.g. span corruption)
Typical use	understanding, embedding	generation	sequence-to-sequence
Flagship model	BERT	GPT	original Transformer, T5

ENCODER-ONLY, DECODER-ONLY, ENCODER-DECODER / 2

- **Encoder-only** (bidirectional self-attention, no causal mask): good at building rich representations of an already-complete input — classification, embeddings, retrieval.
- **Decoder-only** (causal self-attention only): good at generating one token at a time — today's LLMs are almost all decoder-only.
- **Encoder-decoder** (bidirectional encoder + causal decoder + cross-attention): good at mapping one sequence to another — translation, summarization.

WHERE THIS IS GOING

- Session 8 trains all three families into pretrained models — GPT (decoder-only), BERT (encoder-only), T5 (encoder-decoder) — and shows how they are used: decoding, prompting, instruction tuning.
- Session 10 shows this taxonomy is not NLP-specific: the same patterns reappear in vision and multimodal models.
- Nothing about encoder-only, decoder-only, or encoder-decoder is specific to text: what actually distinguishes the three families is which positions are allowed to attend to which other positions, not the domain of the input.

REFERENCES

- Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. Published as a conference paper at ICLR 2015. https://arxiv.org/abs/1409.0473
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. https://arxiv.org/abs/1706.03762
- Graves, A., Wayne, G., & Danihelka, I. (2014). *Neural Turing Machines*. https://arxiv.org/abs/1410.5401
- Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D. F., & Chao, L. S. (2019). *Learning Deep Transformer Models for Machine Translation*. ACL 2019. https://aclanthology.org/P19-1176/
- Beltagy, I., Peters, M. E., & Cohan, A. (2020). *Longformer: The Long-Document Transformer*. https://arxiv.org/abs/2004.05150

REFERENCES / 2


Fleuret, F. (2023). *Deep Learning* (course notes), University of Geneva. `https://fleuret.org/dlc/`, §13.1–13.3. Licensed CC BY-NC-SA 4.0.

License & Attribution

LICENSE & ATTRIBUTION

- **Main source.** All sections except those listed below are adapted from **Deep Learning for NLP (DL4NLP)**, SLDS, LMU Munich — licensed **CC BY 4.0**: `https://creativecommons.org/licenses/by/4.0/`. Chapter 3 (*Transformer*) is the backbone; chapter 2 provides Root 2 (encoder-decoder RNNs, Bahdanau attention), the basic attention recipe, and tokenization. Frames were reordered, some merged or condensed (word-order proof, parameter count, tokenization), and a few retitled.

LICENSE & ATTRIBUTION / 2

- **Fleuret.** The Neural Turing Machine root (Root 1), the permutation-invariance frame, and the pre-norm/post-norm frame are freshly drafted following the framing and citations of **François Fleuret's *Deep Learning* course**, University of Geneva (`https://fleuret.org/dlc/`, §13.1–13.3) — licensed **CC BY-NC-SA 4.0**: non-commercial academic teaching use is permitted, reuse must be attributed and shared under the same license.
- **Original to this course.** The opening "Why Attention?" and "Position-Fixed vs. Content-Based Weighting" frames, the explanatory text of the Bahdanau frames, the sparse-attention patterns frame (figure from DL4NLP), and "The Three Transformer Families" section.